



THE EMOTIONAL RELATIONAL MODEL (ERM)

The Ledger Emotion Has Been Missing

Foundational Architecture • December 2025

SLIDE 1: The Failure Pattern

(Not "uncomfortable truth" - "failure pattern")

Title: "Why Every CX Initiative Dies"

Visual: A graveyard of initiatives

- 2019: "Improve onboarding" ✓ Closed
- 2020: "Reduce support load" ✓ Closed
- 2021: "Fix checkout flow" ✓ Closed
- 2022: "Improve onboarding" ✓ Closed ← **same problem**
- 2023: "Reduce support load" ✓ Closed ← **same problem**

Text:

*"Your organization closed 847 improvement initiatives last year.
This year, you're starting from zero.
Not because you didn't fix things.
Because the system has no memory.*

*You're rebuilding knowledge every quarter.
The system cannot learn."*

Why this works: It's not abstract pain ("we can't measure emotion"). It's concrete, expensive, and infuriating. Every exec has lived this.

SLIDE 2: The Reason Is Structural

Title: "The Substrate That Doesn't Exist"

Visual: Two columns

Column 1: "Everything Else Has a Ledger"

- Money → Double-entry bookkeeping
- Code → Git (versioned, branched, merged)
- Identity → Primary keys (unique, indexed, joined)
- Payments → Stripe intents (state machine, lifecycle)
- Data → Relational model (foreign keys, constraints)

Column 2: "Emotion Doesn't"

- Feedback → ?
- Action → ?
- Outcome → ?
- Memory → ?

Text:

"Emotion is the only enterprise signal with:

- *No identity*
- *No state*
- *No lineage*
- *No memory*

This is not a tooling problem.

This is a missing primitive."

Why this works: Makes the problem feel *categorical*, not incremental. You're not selling "better CX software" - you're identifying the foundational gap.

SLIDE 3: What Missing Infrastructure Looks Like

Title: "The Cost of No Ledger"

Visual: Split screen - two scenarios

Left: "With Infrastructure" (e.g., Git)

Commit A → fixes bug

Commit B → breaks feature

Git bisect → identifies Commit B

Revert → system heals

Memory → preserved forever

Right: "Without Infrastructure" (Current CX)

Initiative A → "improved onboarding"

Customer churn spikes

Team investigates → no lineage

Blame spreads → nothing reverts

Memory → tribal, lost when people leave

Text:

"Every other system can answer:

- *What changed?*
- *When did it break?*
- *What fixed it before?*

CX cannot. Not because teams don't care.

Because the ledger doesn't exist."

Why this works: Makes the absence *expensive* and *visible*. This is the slide where they start to feel it.

SLIDE 4: The Physics of Emotion

Title: "Emotion Is Not Unstructured"

Visual: Before/After comparison

What We Thought Emotion Was:

- Unstructured text
- Sentiment scores
- Themes and word clouds
- Commentary

What Emotion Actually Is:

- **Identity** → Which surface was affected
- **State** → The current emotional condition
- **Transition** → What changed it
- **Lineage** → How it unfolded over time

- **Memory** → What happened when we acted
- **Consequence** → Whether the action held

Text:

*"We treated emotion like commentary.
It's actually a state machine.*

*We never built the schema that matches its physics.
Until now."*

Why this works: The reframe feels like a *discovery*, not an invention. You're revealing what was always true.

SLIDE 5: The Primitive

Title: "The SEU: Emotion's Atomic Unit"

Visual: Show ONE example - make it concrete

EXPERIENCE DRIVER: "Onboarding Clarity"

SEU STATE (Nov 2024):

├ Emotional Valence: -2.3 (anxiety/confusion)

├ Urgency: High

├ Activity: 847 mentions (↑ 34% from Oct)

├ Impact Tier: P1

└ Evidence: 847 customer mentions linked

This is not a theme. This is an account.

One driver. One SEU. One truth.

Text:

*"The SEU is the emotional account of an Experience Driver.
It has state, history, and lineage.
Exactly like:*

- *A bank account (balance, transactions, history)*
- *A UTXO (unspent, spendable, traceable)*
- *A primary key (unique, indexed, joinable)*

Once you have the SEU, emotion becomes computable."

Why this works: ONE concrete example is worth 100 abstract explanations. They need to see the state.

SLIDE 6: The Three Laws

Title: "The ERM: The Model That Makes It Real"

Visual: Show the laws as *constraints*, like database rules

-- Law 1: Identity

UNIQUE(experience_driver_id, window_id)

→ One Driver = One SEU = One Truth

-- Law 2: Lineage

FK(ou_activation_id) → immutable, timestamped

→ Every action is traceable forever

-- Law 3: Memory

FK(seu_id) → outcome_evaluations

→ Past states constrain future actions

Text:

"Three laws. Enforced like double-entry bookkeeping."

These aren't features.

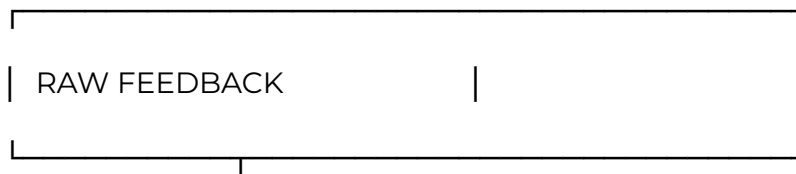
These are the invariants that turn emotion into infrastructure."

Why this works: Shows this is *formal*, not aspirational. The SQL makes it real.

SLIDE 7: The Loop

Title: "PDAM: The Execution Engine"

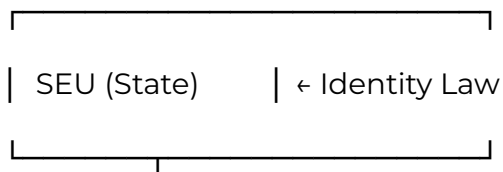
Visual: Show the six stages as ONE continuous loop, not a list



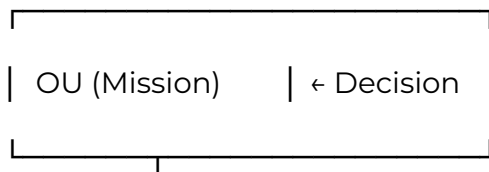
↓

[UNPACK + STRUCTURE]

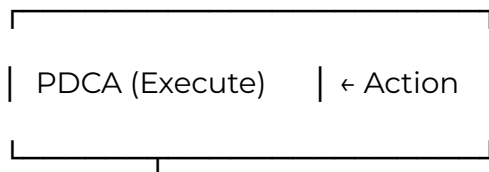
↓



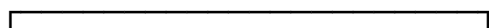
↓



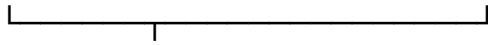
↓



↓



| MEMORY (Learn) | ← Lineage Law



↓

[NEXT CYCLE] → SEU updated with outcome

Text:

*"Six stages. Fully deterministic.
The emotional equivalent of:*

- *Payment lifecycle (Stripe)*
- *Event sourcing (Kafka)*
- *Version control (Git)*

*Emotion is no longer analysis.
It's a closed-loop system."*

Why this works: One visual that shows the whole thing. The loop is the insight.

SLIDE 8: The Proof

Title: "What This Actually Looks Like"

Visual: Show ONE real example end-to-end

DRIVER: "Onboarding Clarity"

SEU STATE (Nov 2024): -2.3, P1, 847 mentions

↓ OU ACTIVATED: "Fix: Simplify signup flow"

↓ PDCA CYCLE: Redesign + test + deploy

↓ OUTCOME EVALUATION:

SEU STATE (Dec 2024): -0.8 (↑ 65% improvement)

Mentions: 312 (↓ 63%)

Elasticity: 1.3 (durable improvement)

MEMORY PERSISTED:

"Last time 'Onboarding Clarity' reached P1,

Action: 'Simplify signup flow' produced 1.3 elasticity.

Recommend same approach if state recurs."

Text:

"From one driver, you can JOIN your way to:

- *The emotional state*
- *The action taken*
- *The outcome achieved*
- *The memory preserved*

This is not a dashboard.

This is a ledger with referential integrity."

Why this works: ONE undeniable example. They can see it working. Not theory - proof.

SLIDE 9: Silent Guidance

Title: "The System Remembers What You Forget"

Visual: Show the memory query in action

NEW ALERT: "Checkout Friction" → P1 (anxiety spiking)

SYSTEM QUERY:

SELECT action, elasticity, outcome

FROM institutional_memory

WHERE seu_driver = "Checkout Friction"

AND priority = "P1"

ORDER BY elasticity DESC

LIMIT 1

RESULT:

"Last time this happened (Q2 2024),

Action: 'One-click guest checkout'

produced 1.7 elasticity.

Recommend reactivation."

Text:

"The system queries its own biography.

Not 'what should we do?'

'What worked last time we saw this exact state?'"

This is institutional memory.

Queryable. Durable. Autonomous."

Why this works: This is the **magic moment**. The system isn't just tracking - it's *learning*. This is when it transcends "better CX tool."

SLIDE 10: What This Enables

Title: "What Becomes Possible"

Visual: Simple list, but each item is a *capability unlock*, not a feature

- ✓ **Agents can act on real emotional state** (not hallucinated summaries)
- ✓ **BI teams can JOIN emotion like any table** (Tableau, PowerBI, SQL)
- ✓ **Execs see cost of emotional failures** (P1 anxiety = \$X revenue risk)
- ✓ **Memory survives turnover** (new PM inherits full lineage)
- ✓ **Fixes are traceable** (what worked, what didn't, why)
- ✓ **Improvements compound** (no more starting from zero)

Text:

*"This is not CX.
This is Emotional Infrastructure."*

Why this works: Short, punchy, undeniable. Each line is a "wait, you can do that?" moment.

SLIDE 11: The Category

Title: "The Ledger Emotion Has Been Missing for 70 Years"

Visual: Timeline

1950s → Relational model (Codd)

1970s → Double-entry accounting standardized

1990s → Version control (CVS → Git)

2000s → Payment intents (Stripe)

2010s → UTXO ledger (Bitcoin)

2020s → Compute/storage separation (Snowflake)

2025 → The Emotional Relational Model

Text:

*"Every foundational system was once invisible.
Then someone saw the primitive.*

*The protocol is defined.
The laws are set.
The schema is stable.
The ledger is live.*

The only question left is:

Who will run the first node?"*

Why this works: Positions this as *inevitable*, not speculative. You're not selling software - you're offering a seat at the foundational moment.
